

Git Commands

Global Settings & Configuration

--global is used to configure Git at the machine level.

For instance, setting the user name and email that would be used while pushing code would only be **required to configure once** after Git installation.

```
git config --global user.name "<user-name>"
git config --global user.email "<email>"
```

Cloning a Project

Clone - Clone acts as the initial setup, it's **done once per project** per computer. It will create a brand new copy of the repository that doesn't exist on our computer yet.

```
git clone <github-repo-link>
```

NOTE: *git clone will automatically initialize and connect our repo URL (setup origin cloud links).*

It's crucial to note that a clone will copy the current instance of the cloud repo, including all the branches and history of commits.

Git Pull

Pull - Download the changes or the latest updates. Git checks the current local branch, compares it with the cloud repo for any new commits, and blends (merges) them directly into the files open in the local repo.

```
git pull <origin-name> <branch-name>
```

Example: The below code will download all the changes/updates from the main branch.

```
git pull origin main
```

💡 **Tip:** Once the tracking link is set up via the `-u` flag, you don't need to type `git pull origin main`. If you are currently standing on the main branch, simply typing `git pull` will update the specific branch you are working on automatically.

Pushing Code from Local Repository to a Blank Repository in the Cloud

The Sequence: Initialize git → Stage files → Commit → Link the Cloud repo → Push

This sequence is only followed the first time a local repo needs to be uploaded to a blank repo in the cloud.

For any future uploads (Push) we just need to: Stage files → Commit → Push

The sequence in detail:

1. `git init` - **Git Initialization** This command shall be used explicitly once per project if not implicitly executed by commands like `git clone`. It creates a hidden `.git` folder that holds all the information related to branches and cloud repository links. This folder keeps track of your history and changes.
2. `git add` - **Stage Files**

This command will stage the file(s) for pushing. `git add .` will stage all the modified and new files in the current project, excluding files/folders specified in the `.gitignore` file.

3. `git commit -m "message"` - **Commit with a message**

Saves a permanent snapshot of your staged changes to your local database.

4. (Optional) - `git branch -M main` - **Force rename the current branch as main**

Ensures your local default branch name matches the default branch name expected by GitHub (main).

5. `git remote add origin <repo-url>` - **Link to the Cloud repo**

This will link the local Git repo with the Cloud repo using the default nickname `origin`.

6. `git push -u origin main` - **Final Push**

The `-u` flag tells Git to create that **upstream tracking link**, so that in the future, you can just type `git push` or `git pull` without extra typing.

The last step might prompt a GitHub authentication, which is covered in the next section.

Sequence at a single glance:

```
# 1. Initialize the repository
git init

# 2. Add files and Commit (Make sure to include a message inside the
quotes!)
git add .
git commit -m "Initial commit"

# 3. Standardize branch name
git branch -M main

# 4. Link to GitHub
git remote add origin https://github.com/pavan-pentyala/bookshelf-api-
learning-project

# 5. Push to GitHub
# When prompted:
#   Username: (Your GitHub username)
#   Password: (Paste your Personal Access Token here)
git push -u origin main
```

Types of Authentication

- **The SSH Key Way** - *Slightly complex, but preferred by professionals*

This uses a pair of cryptographic keys (public and private) which are machine-specific. The public key is shared with GitHub and acts as your permanent digital handshake. It is a *one-time setup per machine* and bypasses browser popups completely.

- **Git GUI (Credential Manager)** - *The easiest and most basic*

This works via HTTPS. When you push, a window pops up asking you to sign in via your web browser. The Windows Credential Manager will securely store this log-in and will not ask for re-authentication for future pushes.

- **Personal Access Token (PAT)** - *Terminal-centric and secure*

This acts as a secure, temporary password replacement generated on GitHub (under Settings → Developer Settings → Personal Access Tokens → Classic). To use it for pushing, you must ensure the `repo` checkbox is checked when creating it.

 **Note:** When pasting your token into the terminal password prompt, **no characters or dots will appear on the screen**. This is normal terminal security! Just paste it once and hit Enter.

Branching in Git - Theory

The Need for Branching: Branching enables keeping the live/working code completely separated from new features and unexpected bugs.

Without branching, trying to implement a new feature directly on the main codebase could break already functional code and disrupt the live application.

With branching, we can spin up an isolated playground based on the latest instance of working code. This allows developers to safely build a new feature, test it thoroughly, and verify its compatibility with existing code.

Similarly, creating a temporary branch to fix a live/production urgent bug enables developers to seamlessly isolate, work on, and test priority hotfixes without the fear of making the production code worse. It allows developers to quickly switch between tasks with a single command based on changing priorities.

Branching creates a pointer based on the current instance of code (usually `main`). Whenever we create a new branch, we can simply switch into it to start working. While cloning `main` into a brand-new folder ("fresh root") on your computer would technically achieve a similar isolation, it introduces major inefficiencies in space and time, as cloning downloads the repository's data all over again. Branching happens locally and instantly.

Note: *In professional tech companies, the `main` branch is literally locked by administrative settings. No single developer—not even the senior ones—can push code directly to `main`. It must go through a separate branch so that automated testing systems and peer reviews can check the code first.*

Git Branching - Commands

Creating a branch - `git branch`

```
git branch <branch-name>

# Example
git branch dark-mode-feature
```

Note - *The above command would only create the branch, and the current working branch would still remain the same. It could cause accidents if you start coding immediately without switching!*

Switching the branch - `git switch` or `git checkout`

```
git checkout <branch-name>

# or
git switch <branch-name>

# Example
git checkout dark-mode-feature
```

Create and Switch in one step To prevent accidents that could be caused by just creating a new branch (as discussed earlier), it is highly advised to create and switch in one single step. Most developers use this instead of running separate commands.

```
git checkout -b dark-mode-feature

# or use the modern switch command with -c parameter (c for create)
git switch -c dark-mode-feature
```

Note - *dark-mode-feature* is the branch that we want to create and jump into. Attempting to create a branch that already exists will throw an error: *fatal: a branch named 'dark-mode-feature' already exists.*

Checking the current branch info - Running `git branch` by itself will list all the existing branches in your local repository, highlighting the branch you are currently standing on.

Adding the `-a` flag can be used to see **all** branches—both on your local machine and in the Cloud. Please note that the cloud branch details are based on your last update (`fetch`) and could be stale if you haven't synced recently.

Using `git branch -a` will give us an output like below:

```
api
backend
* dark-mode-feature
frontend
main
remotes/origin/HEAD -> origin/main
remotes/origin/frontend
remotes/origin/main
```

The asterisk () against `dark-mode-feature` indicates that the user is currently standing on that branch.*

The cloud branches are those that start with the `remotes/` prefix.

Pushing a Local Branch to the Cloud (Upstream Tracking)

When you create a brand-new branch locally, it only exists on your computer. To create its matching twin on GitHub and upload your code, you use the `-u` (or `--set-upstream`) flag. They are completely identical, but `-u` is the shortcut.

```
git push -u origin feature-login

# exact same as:
git push --set-upstream origin feature-login
```

Note - You only need to type `-u origin feature-login` **once** for the first push. Git links them together, and for any future updates, you can just type a simple `git push` while standing on that branch.

Syncing a Cloud Branch Down to Local (Fetch + Switch)

If a teammate creates a new branch on GitHub, it won't automatically show up as an editable branch on your computer. You have to download the blueprints first, and then switch into it.

```
# 1. Download the latest cloud metadata (updates the remotes/ list)
git fetch

# 2. Switch to it. Git sees it in the remotes/ list and automatically
creates your local copy!
git checkout analytics-tracker
```

Deleting a Local Branch - `git branch -d`

Once a feature is finished and successfully merged/uploaded, you should clean up your workspace by deleting the temporary playground.

```
git branch -d dark-mode-feature
```

Note - Git has built-in safety guards. If you have unmerged or unpushed work on that branch, Git will refuse to delete it to protect your work. If you intentionally want to force-delete and throw that code in the trash, use a **capital** `-D`: `git branch -D garbage-test-branch`.

Also, you cannot delete a branch while actively standing inside it! Switch to `main` first, then delete.

Real-World Branching Sequence Workflow

```
# 1. Check where you are currently standing to avoid mistakes
git status
# 2. Switch to main to make sure you are branching off the freshest,
working code
git checkout main
# 3. Pull the absolute latest code from the cloud so your foundation is
up to date
git pull
# 4. Create your new feature branch and jump right into it using the
shortcut
git checkout -b feature-login
# 5. Verify that you successfully moved into the new playground
git branch
```

Git Merging - Theory

Here are your comprehensive **Milestone 3 Theory Notes** formatted cleanly in Markdown. I have structured them to match your clear, practical style, integrating the core mechanics with the exact real-world doubts and scenarios you brainstormed.

What is Merging?

Branching enables us to keep code isolated in parallel universes. However, a feature or hotfix is only useful once it is brought back to the live application. **Merging** is the process of taking the history, commits, and file changes from an isolated branch and cleanly blending them back into another branch (typically `main`).

The Two Git Merge Strategies

Git uses two entirely different strategies to combine your code based on how your project's timeline looks:

1. The "Fast-Forward" Merge (The Smooth Path)

This happens when you branch off `main` to work on a feature, and while you are working, **nobody else on your team touches** `main`. The `main` branch remains frozen in time.

- **How Git handles it:** Because there are no competing updates on `main`, Git doesn't do any complex file blending. It simply slides the `main` marker forward along the timeline until it sits at your latest feature commit.
- **Real-World Impact:** It is instant, effortless, and requires zero file manipulation.

2. The "Three-Way" Merge (The Combined Path)

This happens when your timelines split. You are working on your feature branch, but a teammate finishes their work first and merges it into `main`. The `main` branch has now moved forward with a new commit.

- **How Git handles it:** Git analyzes three data points: the common ancestor (where you split), your latest commit, and `main`'s latest commit. If you and your teammate edited completely different files, Git automatically mixes the code together and creates a special, brand-new snapshot called a **Merge Commit** to tie the split timelines back together.

The Merge Conflict: When Auto-Merge Fails

If two developers edit the **exact same line of code in the exact same file**, Git will freeze the merge. It refuses to guess which version is correct because making a blind guess could break the live application.

A Merge Conflict is not a system crash or an error; it is simply Git safely pausing the process and asking a human to act as the judge. It marks the contested file with special markers (<<<<<<<, =====, >>>>>>>) and waits for you to choose the winning code.

If two developers add completely different functions to the same file, does it conflict?

No. If a teammate adds a function at the top of `index.js` and you add one at the bottom, Git is smart enough to auto-merge them seamlessly. However, **you must still test them together locally** before pushing to production to ensure the two new pieces of logic play nice functionally.

What happens if two Pull Requests are raised at the exact same time?

GitHub manages concurrent PRs on a strict "**First Come, First Served**" basis. Absolutely no one's PR is declined or thrown away.

1. The developer who merges first gets the easiest ride. Their code lands on `main` effortlessly.
2. The developer who attempts to merge *second* gets their PR put on hold. GitHub dynamically shifts their PR button to gray/red and states: *"This branch has conflicts that must be resolved."*
3. The second developer must then fetch the updated `main` branch down to their laptop, merge it into their feature branch locally, resolve the conflict, test it, and push it back up. Once pushed, their GitHub PR button turns green again.

Git Merge - Commands

Step 1: Prepare the Receiving Branch

It is crucial that we are standing on the branch that we want changes merged *into*—which will be `main` most of the time.

Additionally, we must pull the latest changes from GitHub so that our local receiving branch and the cloud repository are perfectly in sync. Skipping this step is a major cause of unnecessary conflicts.

```
git checkout main

# Pull to get the absolute latest version from the cloud
git pull origin main
```

Note: We might not always want to merge into `main`. For example, we might need to merge a feature into a specific release branch like `release-2.0.7`. The process remains exactly the same; you just replace `main` with your target branch name.

Step 2: Trigger the Merge

Run the merge command followed by the name of the branch that holds the new changes you want to bring in.

```
git merge <branch-name>

# Example: We want to merge changes from checkout-feature into main
git merge checkout-feature
```

If the timeline allows for a **Fast-Forward** or an automatic **Three-Way Merge**, Git completes the process instantly, and you are done!

Handling a Merge Conflict

If your changes collide with another developer's changes on the same line, Git will pause and output an error message:

```
Auto-merging index.js
CONFLICT (content): Merge conflict in index.js
Automatic merge failed; fix conflicts and then commit the result.
```

When this happens, Git expects us to manually fix the conflicted files, stage them, and commit. Until we do, **the git merge remains in a paused state**. As soon as the fix and commit are completed, the merge successfully finalizes.

The Emergency Undo (Abort)

What if a massive conflict explodes and you don't have the time or context to fix it right away? You can completely roll back the attempt using the abort command:

```
git merge --abort
```

This completely resets your repository to its exact state before you typed `git merge`. The merge is canceled, and you exit the paused mode.

Resolving a Merge Conflict Step-by-Step

When a merge is paused, your terminal prompt will change from just showing your branch (main) to showing something like (main|merging), indicating Git is waiting on you.

Step 1: Find out what is broken

```
git status
```

Under a section labeled "Unmerged paths", Git will list the conflicted files in bright red text (e.g., both modified: index.js).

Step 2: Open the file and fix the code

Open the conflicted file in your editor (like VS Code). Look for the markers Git injected directly into your code:

```
<<<<<<< HEAD
const theme = 'dark'; // Code on your current local branch (main)
=====
const theme = 'light'; // Code coming from the branch you are trying to
merge (checkout-feature)
>>>>>>> checkout-feature
```

1. Delete the lines of code you do not want to keep.
2. Keep the lines you do want (or combine/stack both lines neatly so both features work).
3. **Crucial:** Manually delete the <<<<<<<, =====, and >>>>>>> marker lines entirely. Your file must look like normal, clean code before saving.

Step 3: Tell Git the conflict is resolved

Once you save your clean file, you must stage it to let Git know it has been repaired:

```
git add index.js
```

Step 4: Finalize the Merge Commit

Now that the conflicted files are staged (turning green in `git status`), you seal the deal by running a standard commit. Because Git knows you are finishing a paused merge, it will auto-generate a merge message for you, so you don't strictly need to add the `-m` flag:

```
git commit
```

(Or write a custom message: `git commit -m "Resolved merge conflict in index.js"`)

The moment you hit enter, your terminal will drop the `|merging` tag, return to normal, and your parallel universes are successfully combined!

Milestone 4: Advanced Git Diagnostics & Housekeeping - Commands

Git Log - `git log`

Git log is used to check all the local commits along with the commits done by other developers until your latest fetch or pull.

Console Lock Tip: If your commit history is long, the console will pause at a colon (:). Pressing the `q` key will instantly quit the log view and return you to your normal terminal prompt.

```
# Generic log command without any filters  
git log
```

Running the generic command will output a detailed breakdown looking like this:

```
commit 3f83340e6820e990637083499ff93ac23637ca90 (HEAD -> main,
origin/main, origin/HEAD, backend, api)
Author: pavan-pentyala <158619773+pavanpentyala@users.noreply.github.com>
Date: Thu Jun 11 08:31:40 2026 +0530

    added teachers file and modified students

commit 36f9813e84fc2e0d5cb739b7eb9602db27089d0b
Author: Pavan Pentyala <158619773+pavanpentyala@users.noreply.github.com>
Date: Thu Jun 11 08:26:43 2026 +0530

    Created courses.txt file
```

Because a real repository can have thousands of commits, we use specific filters on a daily basis to cut through the noise:

1. The One-Liner (`--oneline`)

Displays each commit in a concise, beautifully formatted single line.

```
git log --oneline
```

Output example:

```
3f83340 (HEAD -> main, origin/main, origin/HEAD, backend, api) added
teachers file and modified students
36f9813 Created courses.txt file
0c1d1f1 Initial project files - README md file added
011ba35 Initial project files - student.txt added
d8ad812 created README md file
```

2. Filter by Author

Isolates commits written by a specific developer.

```
git log --author="Pavan"
```

3. History of a Specific File

Shows only the commits that modified a specific file.

```
git log -- <file-name.ext>
```

4. Filter by Text Search (--grep)

Searches for specific keywords inside the commit messages.

```
git log --grep="created"
```

 **Message Filter Note:** Because developers use different phrasing (e.g., "added", "new-file", "implemented") instead of just "created", you must keep these variations in mind when searching with --grep.

5. Filter by Time (--since)

Filters history using natural language or specific calendar dates.

```
git log --since="3 days ago"  
git log --since="2026-06-01"
```

 **Pro-Tip:** You can combine multiple filters together! For example, to see a clean, one-line history of changes to a single file, run: `git log --oneline -- teachers.txt`

Git Blame - git blame

`git blame <file-name>` isolates the author of every single line of code inside a specific file. This is highly useful when you need to find out who introduced a change that is now colliding with your code or causing a bug.

```
git blame index.js
```

- **Line Range Filtering (-L):** If a file has hundreds of lines and you only care about lines 15 through 70, you can isolate them:

```
git blame -L 15,70 index.js
```

- **Ignore Whitespace (-w):** Bypasses purely cosmetic formatting changes (like adding tabs or new empty lines) so you can see who wrote the actual functional logic:

```
git blame -w index.js
```

Git Commit Amend - --amend

The `--amend` option acts as a quick "undo" or modifier for your absolute last commit. It is used when you make a commit but immediately notice a typo in the message, forgot to include a specific file, or forgot to save a configuration update.

```
# 1. Stage the missing or corrected file
git add missed-file.js

# 2. Absorb it into the previous commit (keeps the same commit message)
git commit --amend --no-edit
```

 **THE GOLDEN RULE OF AMENDING:** Only amend a commit if it exists **purely on your local machine**. If you have already pushed your commit to GitHub, do not amend it. Amending changes the commit ID, which will disrupt history and cause sync errors for your teammates.

Git Revert - git revert

If a feature commit from last Tuesday is causing a production crash, you do not erase or delete it from history; you **revert** it.

```
git revert a1b2c3d
```

- **What Git Does:** It instantly calculates the exact mathematical opposite of whatever commit `a1b2c3d` did, applies those reversals to your files, and automatically creates a brand-new commit prefixed with `Revert "Original commit message"`.

- **The Production Emergency Workflow:** You never run a revert directly in the cloud. Instead, you follow this precise safety pipeline:
 1. Pull the freshest code down to your machine: `git pull origin main`
 2. Branch off into a dedicated fix branch: `git checkout -b hotfix-rollback`
 3. Execute the revert locally: `git revert <bad-commit-id>`
 4. Push the fix branch to the cloud: `git push origin hotfix-rollback`
 5. Raise a Pull Request (PR) to merge your hotfix into `main`. This safely removes the broken code for users while allowing you to safely investigate the bug on your laptop.

Additional Concepts

Git rebase

Git rebase and merge might feel similar, and used for bringing in commits from other branch. But they have a clear distinction, and understanding it is very crucial otherwise it could cause chaos.

The Difference : *merge brings incoming branch's history into your current branch. rebase moves your current branch commits so it starts from incoming branch's latest commit.*

The code would be the same in both cases, but the history of commit differs. And how crucial our commit history would help us decide between merge and rebase

Use case : *rebase is useful when we want to fetch the latest from main and integrate it to feature before raising PR*

Git `rebase` is not a simple integration; it means "Move my current branch so it starts from the incoming branch (usually main)"

```

A---B---C---D---E---F---G---H---I---J    (main)
      \
      X---Y                                (feature)

Merge :
git checkout main
git merge feature
A---B---C---D---E---F---G---H---I---J-----M    (main)
      \                                           /
      X-----Y-----/

Rebase :
git checkout feature
git rebase main
A---B---C---D---E---F---G---H---I---J---X'---Y'    (feature)

```

💀 **Danger - Don't ever do this!!!** Rebasing `main` onto `feature` is usually wrong

```

git checkout main
git rebase feature

A---B---C
      \
      X---Y---D'---E'---F'---G'---H'---I'---J'    (main)

```

Git fetch vs Git pull - when to use which

Git pull = Git fetch + Git Merge

`fetch` would retrieve latest commits and data, stores in a hidden, isolated cache folder.

Why use `fetch` when we have `pull` - When we are working on something and it's still in progress, and we wouldn't want that work to be disrupted but we still want to keep our local repo up to date with latest commits from cloud. `pull` would rewrite files and our code might be lost if not committed.

Other reasons to use `fetch` over `pull`

- When we want to keep track of the changes without updating local.
- When we aren't ready to solve merge conflicts

*Merge conflicts are inevitable and unavoidable if there are any; using fetch wouldn't be an escape, we would still need to merge the **"isolated cache folder"**.*

How do we keep track of changes with `fetch` ?

- Using `git log`, we can see that commit changes.

`git log main..origin/main --oneline` - will list the exact commit messages, authors, and hashes of every single commit your teammates pushed to GitHub that you don't have yet. If it returns absolutely nothing, your teammate hasn't pushed anything new

`..` - means "show me what is in the right side (`origin/main`), but not in the left side.

- Using `git diff` for file level inspection

1. `git diff main origin/main --stat` - to inspect which files have been modified, we see how many lines have been modified.

`--stat` - flag gives you a high-level summary. It will list out every file changed, how many lines were added (+), and how many lines were deleted (-).

Sample output :

```
app.js          | 4 +--
utils/auth.js   | 24 ++++++++-----
2 files changed, 28 insertions(+), 10 deletions(-)
```

`diff` can be used for high-level comparison like above between two specific reference points (branches/commits/repos).

2. Line-by-Line code review with colour coding

```
git diff main origin/main app.js
```

This allows you to read their code entirely offline, review it for bugs, and mentally prepare for any merge conflicts before you ever type the word `merge`;

Pro tip : When there are too many line(s) modified or too many files modified `log` and `diff` might feel ineffective, in such cases, we can check the latest main (read-only) using `git switch origin/main --detach`, `--detach` would detach the pointer(HEAD) from main and temporarily place it on top the read-only `origin/main`

when we use `git fetch` our cache folder would also include information on the branches that are on cloud, so if we don't have a branch that exists on cloud, we can simply use `git checkout <branch-name>` and the branch would be created in local.

Under modern Git architecture, you can use the newer, safer command: `git switch <branch-name>`. If Git sees that `<branch-name>` doesn't exist locally but exists exactly on origin, it will automatically create the local branch and configure it to track the cloud repository seamlessly.

Git reset

`git revert` is used when we need to revert back to a previous commit while keeping the commits intact and clear.

`git reset` would do something similar but would erase the history of commits made after the reset point.

`git reset` would never be used on a code that has already been pushed to cloud. It's mainly used on local repositories. Even though we can use `git reset` to rollback a change made to production; we shouldn't because we should keep the commit history in order; we wouldn't want to clear the history that already made it production.

Always use revert if the mistake has already been pushed to GitHub. Because it only adds new commits, your teammates' local repos will seamlessly pull the fix without breaking their timelines

Flavours of `git reset` :

Flag	Explanation
<code>--mixed</code>	would clear the history of commits to the reset point but keep the code open and untouched on screen as uncommitted changes(unstaged); we can review and the modify the code and solve the issue that made us to reset in the first place
<code>--hard</code>	would clear the history of commits and the we would be take back to the reset point everything after the reset point is wiped out permanently
<code>--soft</code>	similar to <code>--mixed</code> but the changes are staged; if we are good with the code, we could simply commit with a message

Pro Tip : *git reset is useful to as a Pre-PR clean-up, we can use `--mixed` or `--soft` flag, clean the slate, and perform a single commit*

Git stash

`git stash` is would create a temporary folder to save our uncommitted code or when we don't to make any ugly/unnecessary commits like "working on hotfix temporary checkpoint"; as it's absurd in professional setting

Let's understand the concept with a scenario, we are working a new feature, and suddenly we were asked to fix a bug in production, `git stash` would save our work as is in a temporary folder. We can create a temporary branch, work on the bug and get back to our unsaved/uncommitted code.

Advanced Professional Safeguard: *By default, running a generic `git stash` only captures **tracked** files. If you have brand-new files sitting in your folder that have never been staged or committed, Git will leave them behind on your screen.*

Pro-Tip Command: *Teach your ledger to use `git stash -u` (Include Untracked). This ensures that completely new files are swept into the stash locker along with your modified files.*

How to use `git stash` like a pro ?

```
# 1. Your code is half-written and messy. An emergency comes in.
git stash

# 2. Your workspace is instantly 100% clean. The mess is safely hidden.
git checkout -b emergency-hotfix
# ...fix the bug, commit it, push it, merge it...

# 3. Go back to your original branch
git switch <branch-name>

# 4. Pop the clipboard locker to explode your messy code back onto your
screen
git stash pop
```

Pro tips

1. `git checkout` **can be used to switch between commits too**, but that would simply place the pointer(HEAD) on a selected commit; it would be read-only, and any changes made are just gone(untraceable). Best way to use this is to branch off from desired commit.
2. `git diff` - **with no arguments would allow us to compare current working files with our last commit**